

The slide features a light gray background with several decorative purple elements: a large circle on the top left, a solid circle in the upper left, a horizontal bar at the top right, a solid circle in the lower right, and a large circle on the bottom right. The title is centered in the upper half of the slide.

PROG IV: Introdução ao Backend com NestJS

Prof^a. Tainá Isabela

Nest.JS

NestJS é uma estrutura Node.js progressiva para criar aplicativos do lado do servidor eficientes, confiáveis e escaláveis. A estrutura oferece suporte total ao TypeScript e faz uso da estrutura Node.js Express, podendo ser utilizado para Programação Orientada a Objetos (**POO**), Programação Funcional (**PF**) e também Programação Funcional Reativa (**PFR**).

NestJS

NestJS combina as melhores práticas de programação e as técnicas de ponta das comunidades de NodeJS tendo muitos conceitos que foram inspirados pelos frameworks mais populares no mundo, como por exemplo: **Angular** e **Spring WebMVC**.

NestJS esconde as complexidades da programação web no NodeJS, fornecendo uma abstração comum do tratamento de requisições web, sendo possível escolher entre **ExpressJs** ou **Fastify** como o mecanismo de fundo. Além disso a estrutura fornece diversas integrações com projetos de terceiros, como **mongoose**, **TypeORM**, entre outros, facilitando a criação de projetos mais dinâmicos.

Conceitos Fundamentais do NestJS

Para entender a fundo o NestJS, vamos explorar alguns conceitos técnicos essenciais:

- **Controllers:** Módulos que recebem e processam requisições HTTP, retornando respostas ao cliente.
- **Services:** Contêm a lógica de negócios e são injetados em controllers, ajudando a separar responsabilidades.
- **Middleware:** Funções que podem ser executadas no ciclo de vida de uma requisição, permitindo manipulações antes de chegar ao controller.

Conceitos Fundamentais do NestJS

- **Guards:** Estruturas que ajudam a proteger rotas e controlar o acesso, determinando se uma requisição deve ser processada.
- **Interceptors:** Permitem adicionar lógica extra em torno da execução de métodos, como logging e transformação de resultados.
- **Pipes:** Facilitam a validação e transformação dos dados que são passados para o controller.

Benefícios do NestJS

O NestJS trás diversos benefícios para a sua aplicação, e alguns deles podem ser vistos na lista abaixo:

- **Simplicidade e Facilidade de Uso:** O NestJS é intuitivo e foi projetado para ser simples de aprender e utilizar. Sua arquitetura clara e bem definida facilita o desenvolvimento, mesmo para iniciantes no mundo do Node.js.
- **Padronização de Projeto e Organização:** O NestJS promove uma robusta padronização de projeto, ajudando a manter o código organizado e legível. Sua estrutura modular e o uso de decoradores mantêm a consistência em todo o projeto.

Benefícios do NestJS

- **Ótima Documentação:** A documentação do NestJS é extensa e bem elaborada, oferecendo muitos exemplos práticos de código. Além disso, fornece guias detalhados para integração com diversas ferramentas, facilitando a implementação de soluções completas e robustas.
- **Comunidade Ativa:** O NestJS possui uma comunidade ativa que contribui constantemente com novos pacotes, extensões e soluções para problemas comuns. Esta comunidade ativa garante muitos recursos e suporte disponíveis para os desenvolvedores.

Benefícios do NestJS

- **Arquitetura Modular:** O NestJS trabalha com módulos ou componentes, permitindo que cada módulo seja responsável por uma funcionalidade específica. Isso torna os projetos altamente organizados, extensíveis e favorece a reutilização de código, facilitando a manutenção e a escalabilidade do sistema.
- **Suporte Total ao TypeScript:** O NestJS oferece suporte completo ao TypeScript, aproveitando todos os benefícios deste superset do JavaScript, como tipagem estática, interfaces e classes. Isso resulta em código mais robusto e menos propenso a erros.

Benefícios do NestJS

- **Escalabilidade, Versatilidade e Progressão:** O NestJS é projetado para ser escalável, capaz de lidar com o crescimento de aplicações tanto em tamanho quanto em complexidade. Sua versatilidade permite que ele seja usado para uma ampla gama de aplicações, desde pequenas APIs até grandes sistemas corporativos. Além disso, sua arquitetura progressiva permite que desenvolvedores adotem novos recursos e melhorias gradualmente, sem grandes refatorações.

Casos de Uso

Alguns cenários práticos para o uso do NestJS:

- **APIs RESTful:** Ideal para construir APIs com uma arquitetura bem definida.
- **Microservices:** A estrutura modular facilita a criação de microserviços que podem ser desenvolvidos independentemente.
- **Aplicações Real-Time:** Com suporte a WebSockets, é possível desenvolver aplicações que necessitam de comunicação em tempo real.

Boas Práticas no Desenvolvimento com NestJS

- **Organização de Módulos:** Sempre busque manter seus módulos bem organizados. Separe as lógicas de cada parte da aplicação para facilitar a manutenção e a escalabilidade.
- **Validação de Dados:** Utilize Pipes para garantir a integridade dos dados que entram no seu sistema.
- **Documentação:** Não esqueça de documentar suas rotas e serviços. O **Swagger** é uma ferramenta que se integra bem com o NestJS para gerar documentação automática.

Instalando e rodando NestJS

Primeiro é necessário instalar NodeJS:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh | bash  
  
\. "$HOME/.nvm/nvm.sh"
```

```
nvm install 24
```

```
node -v # Should print "v24.15.0".
```

```
npm -v # Should print "11.12.1".
```

Instalando e rodando NestJS

Após isso, é necessário instalar o NestJS CLI e com isso criar um projeto padrão:

```
npm i -g @nestjs/cli  
nest new project-name
```

O projeto será criado com o node modules e um diretório src/ que será preenchido com vários arquivos principais que iremos abordar logo abaixo. Para rodar o projeto basta:

```
npm run start
```

Instalando e rodando NestJS

Um dos pontos mais fortes para o uso de NestJS é a sua organização que é estruturada e intuitiva, feita através de Model-Service-Controller, onde cada um tem sua responsabilidade evitando um código poluído e aumentando a produtividade.

Quando iniciamos um projeto NestJS padrão, comumente é gerado alguns arquivos básicos para o funcionamento do framework:

```
.
├── LICENSE
├── nest-cli.json
├── package.json
├── package-lock.json
├── README.md
├── src
│   ├── app.controller.spec.ts
│   ├── app.controller.ts
│   ├── app.module.ts
│   ├── app.service.ts
│   └── main.ts
├── test
│   ├── app.e2e-spec.ts
│   └── jest-e2e.json
├── tsconfig.build.json
└── tsconfig.json
```

Instalando e rodando NestJS

Dentro do diretório de `/src` já é possível ver como funciona a organização Model-Service-Controller que o NestJS implementa.

- `src/main.ts` é o arquivo de entrada desta aplicação.
- `app.module.ts` é um módulo Nestjs que é usado para organizar códigos na visão lógica.
- O `app.service.ts` é usado para manipular a lógica de negócios.

Instalando e rodando NestJS

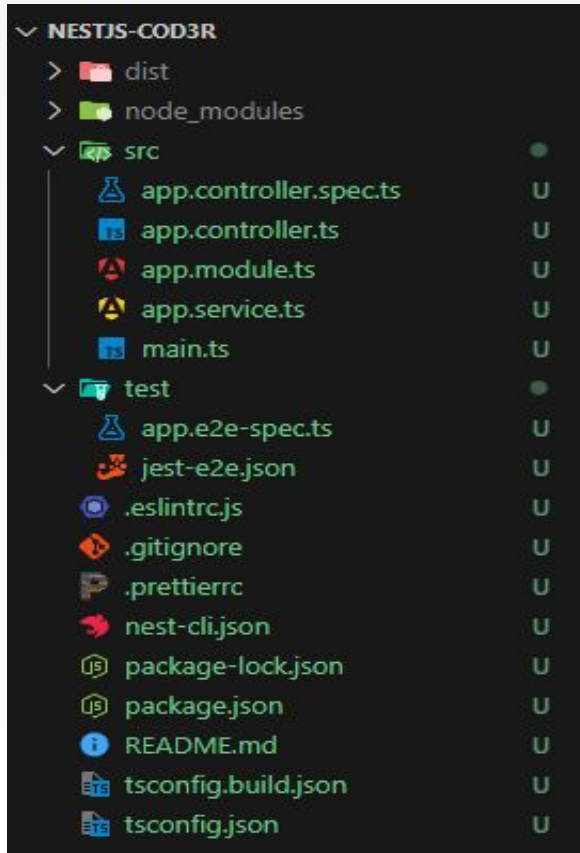
- O `app.controller.ts` é o controlador do MVC, para lidar com a solicitação recebida, e responder o resultado manipulado de volta ao cliente.
- O `app.controller.spec.ts` é um arquivo de teste para `app.controller.ts`. obs: Nestjs usa Jest como framework de teste.
- A pasta de teste é para armazenar arquivos de teste e2e (end-to-end ou ponta-a-ponta).

Instalando e rodando NestJS

Embora neste caso não tenhamos um arquivo específico para a modelagem de alguma entidade, o NestJS possibilita uma maior facilidade na hora de se criar um novo CRUD:

- `nest g resource entity`

Conhecendo o Projeto



Pastas:

- **dist:** A pasta dist é onde o código JavaScript transpilado a partir do código TypeScript da aplicação é armazenado.
- **node_modules:** A pasta node_modules contém todas as dependências do projeto, ou seja, todos os pacotes de terceiros que a aplicação precisa para funcionar.
- **src:** A pasta src é onde você coloca o código-fonte principal da aplicação. Este é o local onde você define a estrutura, lógica e funcionalidade da sua aplicação.
- **test:** A pasta test é onde você coloca os testes da sua aplicação. Em um projeto NestJS, testes são uma parte importante para garantir que o código funcione conforme o esperado.

Arquivos:

- app.controller.spec.ts:
 - Contém testes unitários para app.controller.ts.
- app.controller.ts:
 - Lida com as solicitações HTTP. Decorado com @Controller, geralmente contém métodos decorados com @Get, @Post, para receber requisições GET e POST, respectivamente. É esse arquivo que permite que algo seja retornado, ao acessarmos `http://localhost:3000/`. Por padrão, ele chama a camada de serviço que retorna uma string "Hello World!".
- app.service.ts:
 - Contém a lógica de negócios da aplicação e pode ser injetado em controladores via injeção de dependência.

Arquivos:

- app.module.ts:
 - O módulo principal da aplicação. Decorado com @Module, é usado para organizar e configurar outros módulos, controladores e provedores (services, repositories, etc.) da aplicação.
- main.ts:
 - Arquivo de entrada da aplicação. Cria uma instância da aplicação NestJS usando o NestFactory e inicia o servidor na porta especificada.
- .gitignore:
 - Especifica os arquivos e pastas que devem ser ignorados pelo Git durante o versionamento.
- Nest-cli.json:
 - Arquivo de configuração para o Nest CLI. Define configurações relacionadas à geração de código e outras opções específicas do Nest CLI.

Arquivos:

- **Package.json:**
 - Contém informações sobre o projeto, dependências, scripts de execução, entre outras configurações.
- **Tsconfig.json:**
 - Arquivo de configuração do TypeScript. Define as opções de compilação.
- **Tsconfig.build.json:**
 - Arquivo de configuração do TypeScript específico para o modo de construção (build). Define opções de compilação para o código que será executado em produção.

Arquivos:

- `.prettierrc`:
 - Arquivo de configuração para o Prettier, uma ferramenta de formatação de código. Define regras de formatação para manter o código consistente.
- `.eslintrc.js`:
 - Arquivo de configuração para o ESLint, uma ferramenta de linting para JavaScript e TypeScript. Define regras para manter um código limpo e consistente.

Arquitetura

A arquitetura do NestJS é baseada em módulos. A aplicação é composta por diversos módulos, formando um conjunto dentro do módulo principal, o `app.module`. Um módulo é uma unidade de organização que agrupa vários componentes relacionados (como controladores e provedores).

Os módulos ajudam a estruturar a aplicação em partes coesas e reutilizáveis, promovendo uma arquitetura modular. Os módulos possuem uma estrutura básica, geralmente incluindo arquivos de configuração (`module.ts`), controladores (`controller.ts`), serviços (`service.ts`) e arquivos de testes.

No entanto, não há obrigatoriedade de incluir todos esses arquivos em um módulo, mas ele só será considerado um módulo se tiver o arquivo de configuração (`module.ts`).

Arquitetura

O padrão de nomenclatura é simples e segue a estrutura: [nome do módulo].[tipo do arquivo].extensão. Por exemplo:

- `pessoa.controller.ts`
- `pessoa.service.ts`
- `pessoa.module.ts`

Neste caso, fica explícito que se trata da estrutura básica de um módulo chamado "pessoa". Embora esses nomes sejam recomendados para manter a clareza e a consistência, eles não são estritamente necessários para o funcionamento, desde que sejam referenciados corretamente onde for necessário.

Módulos

Para compreender como o NestJS funciona e como desenvolver nele, é essencial conhecer o funcionamento dos módulos, como se comunicam e como configurá-los. Um módulo no NestJS é simplesmente uma classe anotada com `@Module`, que pode receber alguns parâmetros de configuração: `imports`, `controllers`, `providers` e `exports`.

Cada aplicação possui pelo menos um módulo, o módulo raiz. O módulo raiz é o ponto de partida que o Nest usa para construir a aplicação, estabelecendo a estrutura interna de dados que o Nest utiliza para resolver relacionamentos e dependências de módulos e provedores.

Módulos

É possível ter apenas um módulo na aplicação inteira? Sim, mas essa não é uma escolha recomendada. A documentação do NestJS enfatiza que os módulos são uma forma eficaz de organizar seus componentes. Portanto, a maioria das aplicações utilizará múltiplos módulos, cada um encapsulando um conjunto de funcionalidades relacionadas.

Parâmetros do `@Module`:

- **Providers:** Provedores que serão instanciados pelo injetor de dependências do Nest e que podem ser compartilhados, pelo menos, dentro deste módulo.
- **Controllers:** Conjunto de controladores definidos neste módulo que devem ser instanciados.
- **Imports:** Lista de módulos importados que exportam os provedores exigidos neste módulo.

Módulos

Parâmetros do @Module:

- Exports: Exporta provedores, permitindo que um provedor seja utilizado em outro módulo sem importar diretamente o provedor, mas apenas o módulo que o expõe.

Podemos gerar novos módulos com o comando:

nest g module [nome]

CRUD REST API

O NestJS nos permite criar manualmente arquivo por arquivo ou gerar arquivos específicos usando a CLI do Nest. No entanto, há uma forma ainda mais interessante: gerar um CRUD completo. Como fazer isso?

Usando a CLI do Nest, basta executar o seguinte comando:

nest generate resource pessoa

Esse comando gera um "recurso" chamado "pessoa". Após rodar esse comando, a CLI pergunta:

```
? What transport layer do you use? (Use arrow keys)
> REST API
GraphQL (code first)
GraphQL (schema first)
Microservice (non-HTTP)
WebSockets
```

CRUD REST API

Você deve escolher a "camada de transporte" que deseja que o Nest use para criar o recurso. Neste caso, usaremos REST API, que provavelmente é a mais comum.

Selecionando a opção REST API, uma nova pergunta surge:

```
? Would you like to generate CRUD entry points? (Y/n)
```

Basicamente, "CRUD entry points" se refere à entidade "pessoa" e aos DTOs de criação e atualização.

Ao confirmar, o console exibe o seguinte ao gerar o recurso "pessoa":

CRUD REST API

```
CREATE src/pessoa/pessoa.controller.ts (959 bytes)
CREATE src/pessoa/pessoa.controller.spec.ts (596 bytes)
CREATE src/pessoa/pessoa.module.ts (264 bytes)
CREATE src/pessoa/pessoa.service.ts (661 bytes)
CREATE src/pessoa/pessoa.service.spec.ts (478 bytes)
CREATE src/pessoa/dto/create-pessoa.dto.ts (33 bytes)
CREATE src/pessoa/dto/update-pessoa.dto.ts (181 bytes)
CREATE src/pessoa/entities/pessoa.entity.ts (24 bytes)
UPDATE src/app.module.ts (326 bytes)
```

CRUD REST API

Ele criou diversos arquivos e também atualizou o `app.module.ts`. Por quê? Porque é necessário declarar ao módulo raiz (`app.module.ts`) que estamos criando um novo módulo na aplicação. A mudança é simplesmente importar o módulo "pessoa" dentro do `app.module.ts` e adicioná-lo ao array de imports, assim:

```
import { Module } from '@nestjs/common';
import { AppController } from './app.controller';
import { AppService } from './app.service';
import { PessoaModule } from './pessoa/pessoa.module';

@Module({
  imports: [PessoaModule],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

Services

Existe um uso constante do PessoaService. Geralmente, usamos o conceito de Service para encapsular regras de negócio, e é aí que os serviços entram. A camada de controller serve apenas para receber a requisição e enviar os dados para o service.

Se houver algum tratamento nesses dados, é responsabilidade do service realizar esse tratamento. Basicamente, um service é uma classe injetável via injeção de dependência do Nest, usada para validar e tratar dados.

Se você tivesse uma conexão com um banco de dados, chamaria essa conexão após passar pelo service. O fluxo seria basicamente:

controller $\Rightarrow$ service $\Rightarrow$ banco de dados.

Services

Os services devem conter a anotação `@Injectable()` para funcionarem perfeitamente. O Nest consegue inferir um injetável sem a anotação, mas isso não é recomendado.

Podemos também gerar services com o comando:

nest g service [nome]

Entidade e Dto

No contexto do NestJS, especialmente quando se utiliza o TypeORM (um ORM para TypeScript e JavaScript), uma entidade é uma classe que mapeia uma tabela do banco de dados.

Cada instância dessa classe representa uma linha da tabela. Existem muitas definições sobre o que é uma entidade em diferentes contextos, mas neste contexto específico, foque na explicação acima.

Entidade e Dto

Um DTO (Data Transfer Object) é um objeto usado para transferir dados entre processos. No NestJS, os DTOs são usados principalmente para definir a estrutura dos dados que serão recebidos nas requisições HTTP e os dados que serão enviados nas respostas HTTP.

Conclusão

O NestJS oferece uma abordagem estruturada e eficiente para o desenvolvimento de aplicativos back-end em Node.js, com sua integração com TypeScript e uma arquitetura modular robusta. Sua documentação abrangente e comunidade ativa tornam a aprendizagem e o suporte acessíveis.

Com recursos avançados e uma ampla gama de extensões, o NestJS é uma escolha sólida para projetos de qualquer tamanho, promovendo desenvolvimento mais rápido e seguro.

Questionário

